Day 151 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 5 reference

Watch: Kishan Manani - Feature Engineering for Time Series Forecasting (PyData London 2022) | Cross-Validation for Time Series Forecasting, Python Tutorial (Egor Howell)

`day151_feature_engineering.py`, segment by segment, in the order `main()` uses things. The feature builders, least squares, the two forecast strategies, nearest neighbours and the two kinds of cross-validation are new, and are copied unchanged into today's exercise file. Day 147's data loaders and Day 149's ARIMA parts are imported from their exercise files. pandas and statsmodels are used only to check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: y = the series · X = the feature matrix (one row per month) · h = months ahead · w = window length · b = fitted coefficients

Segment 0 — Setup

Imports and the candidate feature sets

CODE

```
import pandas as pd
from statsmodels.tsa.ar_model import AutoReg
warnings.filterwarnings("ignore")
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day147_exercise as d147
    import day149_exercise as d149
M = 12
SPECS = {
    "lags 1-13": dict(lags=tuple(range(1, 14))),
    "lags 1, 12, 13": dict(lags=(1, 12, 13)),
    "trend + months": dict(trend=True, calendar="months"),
    "lags 1, 12, 13 + trend + months": dict(lags=(1, 12, 13), trend=True, calendar="months"),
    "lags 1, 12, 13 + trend + 2 waves": dict(lags=(1, 12, 13), trend=True, calendar=2),
    "lags 1, 12, 13 + trend + months + 12-month mean, sd": dict(lags=(1, 12, 13), trend=True,
        calendar="months",
    rolling=((12, "mean"), (12, "sd"))),
}
```

- `from statsmodels.tsa.ar_model import AutoReg` — statsmodels' autoregression, the reference for least squares on lags.
- `SPECS = {"lags 1-13": dict(...), ...}` — a dictionary of dictionaries: name → the settings of one feature set.
- `tuple(range(1, 14))` — the lags 1 to 13 as a tuple.
- `rolling=((12, "mean"), (12, "sd"))` — a tuple of (window, statistic) pairs.

Segment 1 — The parts

lag and rolling

MATH

$$\text{lag } k\text{: } x_t = y_{t-k};\quad \text{rolling: } x_t = \text{stat}(y_{t-s-w+1}, \ldots, y_{t-s}),\ s = \text{shift}$$

CODE

```
def lag(y, k):
    out = np.full(len(y), np.nan)
    out[k:] = y[:len(y) - k]
    return out
def rolling(y, w, stat, shift=1):
    out = np.full(len(y), np.nan)
    win = np.lib.stride_tricks.sliding_window_view(y, w)
    first = w - 1 + shift
    out[first:] = stat(win, axis=1)[:len(y) - first]
    return out
```

- `np.full(len(y), np.nan)` — an array of NaN (not a number): unknown until filled.
- `out[k:] = y[:len(y) - k]` — slice assignment: the last n – k places get the first n – k values.
- `np.lib.stride_tricks.sliding_window_view(y, w)` — every window of w consecutive values as a row, without copying.
- `stat(win, axis=1)` — the function passed in (np.mean or np.std) applied along each row.
- `out[first:] = ...[:len(y) - first]` — the first windows land on the months they precede.

month_dummies and fourier

MATH

$$\text{Fourier pair } j\text{: } \sin(2\pi\, j\, m / 12),\ \cos(2\pi\, j\, m / 12)$$

CODE

```
def month_dummies(months):
    mo = months.astype("datetime64[M]").astype(int) % M
    return (mo[:, None] == np.arange(1, M)).astype(float)
def fourier(months, K):
    mo = months.astype("datetime64[M]").astype(int) % M
    k = np.arange(1, K + 1)
    return np.column_stack([np.sin(2 * np.pi * mo[:, None] * k / M), np.cos(2 * np.pi * mo[:,
        None] * k / M)])
```

- `months.astype("datetime64[M]").astype(int) % M` — months since 1970-01 as an integer; % 12 gives 0 for January.
- `mo[:, None] == np.arange(1, M)` — broadcasting: a column against a row gives a months-by-11 table of True/False.
- `.astype(float)` — True/False become 1.0/0.0.
- `mo[:, None] * k / M` — every month times every wave number, as one table.

make_features: one function for every horizon

CODE

```
def make_features(y, start, spec, h=1):
    months = np.datetime64(start, "M") + np.arange(len(y))
    cols, names = [np.ones(len(y))], ["intercept"]
    for k in spec.get("lags", ()):
        cols.append(lag(y, k + h - 1)); names.append(f"lag {k + h - 1}")
    for w, stat in spec.get("rolling", ()):
        cols.append(rolling(y, w, np.mean if stat == "mean" else np.std, shift=h));
            names.append(f"{stat} of {w}, ending {h} back")
    if spec.get("trend"):
        cols.append(np.arange(len(y)) / 120.0); names.append("trend")
    cal = spec.get("calendar")
    if cal == "months":
        D = month_dummies(months); cols += list(D.T); names += [f"month {j + 1}" for j in range(1,
            M)]
    elif cal:
        D = fourier(months, cal); cols += list(D.T); names += [f"wave {j}" for j in
            range(D.shape[1])]
    return np.column_stack(cols), names
```

- `np.datetime64(start, "M") + np.arange(len(y))` — a month plus an integer array: every row's calendar month.
- `spec.get("lags", ())` — dict.get with a default: an empty tuple when the set has no lags.
- `cols.append(...); names.append(...)` — two statements on one line, separated by a semicolon.
- `cols += list(D.T)` — D.T is the transpose; list() turns its rows (the original columns) into separate arrays.
- `elif cal:` — any other truthy value: a number of Fourier waves.

fit_ols and the two forecast strategies

MATH

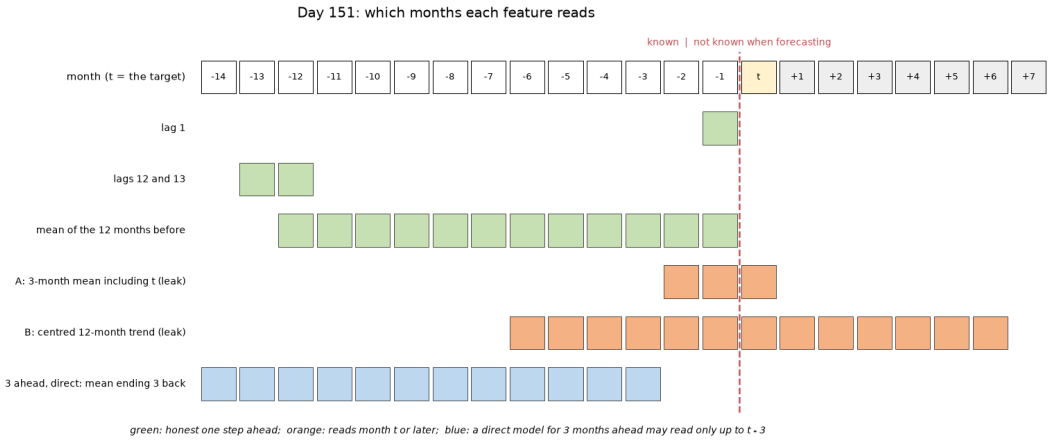
$$\textit{recursive: y-hat}_{T+1} = x_{T+1}\, b\textit{, fed back in;}\quad \textit{direct: y-hat}_{T+h} = x_{T+h}^{(h)}\, b_h$$

CODE

```
def fit_ols(X, y):
    ok = ~np.isnan(X).any(axis=1) & ~np.isnan(y)
    return np.linalg.lstsq(X[ok], y[ok], rcond=None)[0]
def recursive_forecast(y, start, spec, b, h):
    yy = np.r_[y, np.full(h, np.nan)]
    for i in range(len(y), len(y) + h):
        X, _ = make_features(yy[:i + 1], start, spec)
        yy[i] = X[-1] @ b
    return yy[len(y):]
def direct_forecast(y, start, spec, h):
    out = np.empty(h)
    for j in range(1, h + 1):
        X, _ = make_features(y, start, spec, h=j); b = fit_ols(X, y)
        Xf, _ = make_features(np.r_[y, np.full(j, np.nan)], start, spec, h=j)
        out[j - 1] = Xf[-1] @ b
```

- `~np.isnan(X).any(axis=1)` — ~ flips True and False: rows with no missing feature.
- `np.linalg.lstsq(X[ok], y[ok], rcond=None)[0]` — least squares; [0] keeps the coefficients.
- `np.r_[y, np.full(h, np.nan)]` — the data followed by h empty places, filled one at a time.
- `X[-1] @ b` — the last row times the coefficients: one forecast.
- `make_features(..., h=j)` — a keyword argument: the same builder, for a model j months ahead.

SYSTEM DIAGRAM



Which months each feature reads.

nearest neighbours and cross-validation

CODE

```
def knn_fit_predict(Xtr, ytr, Xte, k=5):
    mu, sd = Xtr.mean(axis=0), Xtr.std(axis=0) + 1e-12
    A, B = (Xtr - mu) / sd, (Xte - mu) / sd
    d = ((B[:, None, :] - A[None, :, :]) ** 2).sum(axis=2)
    return ytr[np.argsort(d, axis=1)[:, :k]].mean(axis=1)
def cv_random(X, y, fit_predict, folds=5, seed=0):
    idx = np.random.RandomState(seed).permutation(len(y)); err = []
    for f in range(folds):
        te = idx[f::folds]; tr = np.setdiff1d(idx, te)
        err.append(np.abs(fit_predict(X[tr], y[tr], X[te]) - y[te]))
    return np.mean(np.concatenate(err))
def cv_time(X, y, fit_predict, folds=5):
    edges = np.linspace(len(y) // 2, len(y), folds + 1).astype(int); err = []
    for a, b in zip(edges[:-1], edges[1:]):
        err.append(np.abs(fit_predict(X[:a], y[:a], X[a:b]) - y[a:b]))
    return np.mean(np.concatenate(err))
```

- `B[:, None, :] - A[None, :, :]` — broadcasting to a 3-D array: every test row minus every training row.
- `np.argsort(d, axis=1)[:, :k]` — for each test row, the positions of the k smallest distances.
- `np.random.RandomState(seed).permutation(len(y))` — a shuffled order of the rows, the same for the same seed.
- `idx[f::folds]` — every folds-th element from position f: one fold of the shuffled rows.
- `np.setdiff1d(idx, te)` — the rows not in the fold.
- `zip(edges[:-1], edges[1:])` — consecutive pairs of block edges: (start, end) of each block.

Segment 2 — The experiments

demo_build: comparing with pandas

CODE

```
gap = lambda a, b: np.nanmax(np.abs(a - b)) if np.array_equal(np.isnan(a), np.isnan(b)) else
    np.inf
print(f"lag 12 against shift(12): largest difference {gap(lag(yc, 12),
    s.shift(12).to_numpy()):.1e}")
print(f"mean of the 12 months before against rolling(12).mean().shift(1): {gap(rolling(yc, 12,
    np.mean), s.rolling(12).mean().shift(1).to_numpy()):.1e}")
print(f"sd of the 12 months before against rolling(12).std(ddof=0).shift(1): "
      f"{gap(rolling(yc, 12, np.std), s.rolling(12).std(ddof=0).shift(1).to_numpy()):.1e}")
ref = pd.get_dummies(pd.DatetimeIndex(tc.astype("datetime64[D]")).month).to_numpy(float)[:, 1:]
```

- `lambda a, b: ... if ... else np.inf` — a one-line function with a conditional expression: infinite when the missing values differ.
- `np.nanmax(...)` — the largest value, ignoring NaN.
- `s.rolling(12).mean().shift(1)` — pandas' version: a 12-month mean, moved one month later.
- `pd.get_dummies(...)[:, 1:]` — pandas' month columns; [:, 1:] drops January's.

demo_leaks: a function inside a function

CODE

```
def score(name, Xf, keep=None):
    tr, te = one_step_rows(Xf, yc, n)
    if keep is not None: te = te & keep
    b = fit_ols(Xf[tr], yc[tr]); rows.append((name, np.mean(np.abs(Xf[tr] @ b - yc[tr])),
        np.mean(np.abs(Xf[te] @ b - yc[te])), int(te.sum())))
    return b
```

- `def score(...):` — defined inside demo_leaks, so it can read yc, n and rows from there.
- `keep=None` — a default argument; None means 'score every test month'.
- `te = te & keep` — & on two True/False arrays: True only where both are.
- `rows.append((name, ...))` — the list belongs to demo_leaks; score adds to it without returning it.

demo_cv: measuring the nearest month in time

CODE

```
Z, pos = X[tr], np.flatnonzero(tr); idx = np.random.RandomState(0).permutation(len(Z)); gaps = []
for f in range(5):
    te_ = idx[f::5]; tr_ = np.setdiff1d(idx, te_); mu, sd = Z[tr_].mean(0), Z[tr_].std(0) + 1e-12
    dist = ((((Z[te_] - mu) / sd)[:, None, :] - ((Z[tr_] - mu) / sd)[None, :, :]) **
        2).sum(axis=2)
    gaps.append(np.abs(pos[tr_][np.argmin(dist, axis=1)] - pos[te_]))
g = np.concatenate(gaps)
```

- `np.flatnonzero(tr)` — the positions (months) of the True entries.
- `te_ = idx[f::5]` — the same folds as cv_random, so the measurement describes those folds.
- `np.argmin(dist, axis=1)` — for each held-out month, the nearest training row.
- `pos[tr_][...] - pos[te_]` — fancy indexing twice: back from rows to months, then the gap in time.

validate and demo_choose

CODE

```
def validate(y, start, spec, origins, h):
    err = []
    for o in origins:
        X, _ = make_features(y[:o], start, spec); b = fit_ols(X, y[:o])
        err.append(np.abs(recursive_forecast(y[:o], start, spec, b, h) - y[o:o + h]))
    return np.mean(err)
val, test = {}, {}
for name, spec in SPECS.items():
    val[name] = validate(yc, tc[0], spec, origins, 24)
    X, _ = make_features(yc[:n], tc[0], spec); test[name] = recursive_forecast(yc[:n], tc[0],
        spec, fit_ols(X, yc[:n]), 24)
    print(f"{name:>52} {val[name]:11.3f} {np.mean(np.abs(test[name] - yc[n:])):13.3f}")
best = min(val, key=val.get)
```

- `val, test = {}, {}` — two empty dictionaries in one line.
- `np.mean(err)` — the mean over every origin and every horizon of a list of arrays.
- `min(val, key=val.get)` — the key whose value is smallest: the feature set with the lowest error.

demo_rolling: did the ranking hold?

CODE

```
later = {name: validate(yc, tc[0], sp, range(first, n - 24 + 1), 24) for name, sp in
    SPECS.items()}
print("did the validation ranking (1975-1988 origins) hold in 1990-1998? mean error over 24
    months, recursive:")
for name in sorted(val, key=val.get):
    print(f"  {name:>52}  validation {val[name]:.3f}  1990-1998 {later[name]:.3f}")
order = sorted(later, key=later.get)
```

- `{name: ... for name, sp in SPECS.items()}` — a dictionary comprehension: one entry per feature set.
- `sorted(val, key=val.get)` — the names, ordered by their validation error.
- `order.index(best) + 1` — where the chosen set ranks later, counting from 1.

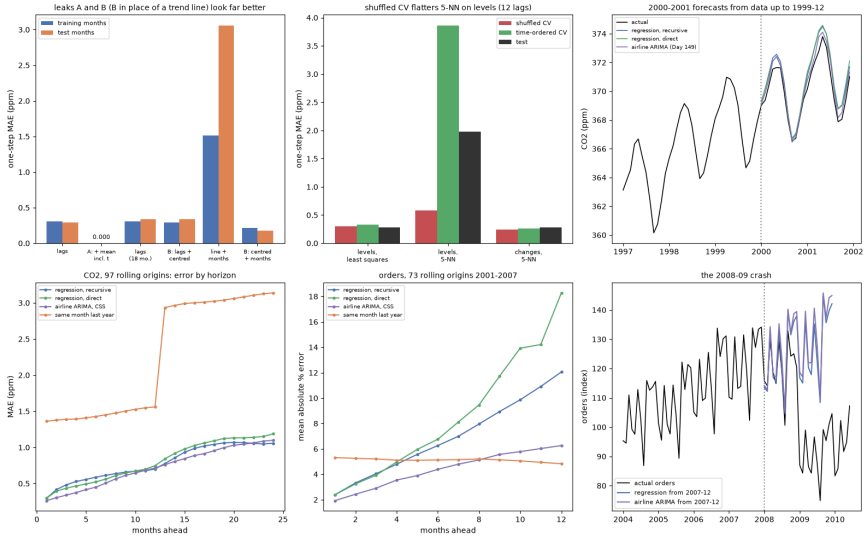
demo_orders: helpers made on the spot

CODE

```
le = np.log(ye); idx = lambda m: int(np.flatnonzero(te == np.datetime64(m))[0])
origins = range(idx("1998-01"), idx("2000-01") + 1)
val = {name: validate(le, te[0], spec, origins, 12) for name, spec in SPECS.items()}
top3 = sorted(val, key=val.get)[:3]
print(f"validation's choice ranked {order.index(best) + 1} of {len(order)} in 2001-2007; the best
    there: {order[0]}; "
    f"validation's top three (all with a trend line: {all(SPECS[k].get('trend') for k in top3)}) "
    f"ranked {', '.join(str(order.index(k) + 1) for k in top3)}")
```

- `idx = lambda m: int(np.flatnonzero(...)[0])` — a small function: the position of a month in the series.
- `sorted(...)[:3]` — the first three of the sorted names.
- `all(SPECS[k].get('trend') for k in top3)` — True only if every one of the three has a trend.
- `', '.join(str(...) for k in top3)` — the three ranks as text, separated by commas.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.full(n, np.nan)` — an array of n missing values.
- `sliding_window_view(y, w)` — every window of w values as a row.
- `a[:, None] == b` — broadcasting a column against a row.
- `d.get(key, default)` — a dictionary value, or the default if the key is missing.
- `~mask; a & b` — not; element-wise and, on True/False arrays.
- `idx[f::k]` — every k-th element, starting at f.
- `min(d, key=d.get)` — the key with the smallest value.
- `{k: f(v) for k, v in d.items()}` — a dictionary comprehension.
- `all(gen)` — True if every value is truthy.

End of Day 151 syntax reference. For the experiments and what they showed, see the Day 151 complete guide.